

LabSoldier 作業指南書 — takebayashi (在室画面フロント エンド担当)

Hackathon Soldier Team

2026-05-21

目次

はじめに	2
第 1 部 共通の基礎知識	2
1.1 アプリの全体像	2
1.2 TypeScript の最低限	2
第 2 部 React の基礎	3
2.1 コンポーネント	3
2.2 useState (状態を持つ)	4
2.3 useEffect (タイミグ処理)	4
2.4 リスト描画と key	5
2.5 条件分岐	5
2.6 イベントハンドラ	5
2.7 カスタムフック	6
第 3 部 担当範囲と作成物	6
3.1 担当範囲	6
3.2 完成イメージ	6
第 4 部 実装手順	7
ステップ 0: ブランチを切る	7
ステップ 1: 既存コードを読む	7
ステップ 2: PresenceList を整える	7
ステップ 3: Character.tsx を画像対応に	9
ステップ 4: usePresencePing にリトライを追加	10
ステップ 5: ManualCheckin の改善	11
ステップ 6: PWA アイコンを配置	12
ステップ 7: スタイルの仕上げ	13
ステップ 8: 動作確認	13

ステップ 9: PR を出す	13
第 5 部よくある詰まりどころ	14
画面が真っ白	14
API リクエストが 401	14
画像が表示されない	14
useEffect が無限ループ	14
CSS が効かない	14
第 6 部ここまでやったら他の人と合流	14
付録 A: チートシート	15
付録 B: 用語集	15

はじめに

この PDF は takebayashi が 在室画面フロントエンドを実装するための指南書です。React の基礎から、コンポーネントの作り方、API 連携、PWA 設定まで一通り解説します。

第 1 部共通の基礎知識

1.1 アプリの全体像

[ブラウザ React] ⇄ [Node.js Express] ⇄ [DB]
frontend/ ← あなたの主戦場

ローカル: frontend は http://localhost:5173

1.2 TypeScript の最低限

```
// 変数と型
const name: string = "takebayashi";
const count: number = 0;
const isOpen: boolean = true;

// 関数
function greet(name: string): string {
  return `Hello, ${name}`;
}

// アロー関数
```

```

const greet = (name: string): string => `Hello, ${name}`;

// オブジェクト型
interface User {
  id: string;
  name: string;
  avatarId: string;
}

// 配列
const users: User[] = [];

// オプショナル
interface Maybe {
  required: string;
  optional?: number;    // 無くてもいい
}

// ユニオン型
type Status = "online" | "offline";

// 非同期
async function load(): Promise<User[]> {
  const res = await fetch("/api/users");
  return res.json();
}

```

第 2 部 React の基礎

2.1 コンポーネント

「画面の部品」を関数として定義する。返すのは JSX/TSX。

```

// 何も受け取らない
function Header() {
  return <h1>LabSoldier</h1>;
}

// props を受け取る
function Greeting({ name }: { name: string }) {
  return <p>Hello, {name}</p>;
}

```

```

}

// 使い方
<Greeting name="takebayashi" />

```

2.2 useState (状態を持つ)

ボタンを押したら何かが変わる、入力欄に文字を入れたら反映される、というのを実現する。

```

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>カウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}

```

- ・ useState(初期値) を呼ぶと [現在値, 更新関数] のペアが返る
- ・ setCount(新しい値) を呼ぶと React が画面を再描画してくれる

2.3 useEffect (タイミング処理)

「コンポーネントが表示されたとき」や「何かが変わったとき」に処理を走らせる。

```

import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState<User[]>([]);

  useEffect(() => {
    // 初回表示時に 1 回だけ走る
    fetch("/api/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []); // ← 依存配列が空 = 初回のみ

  return (
    <ul>

```

```

    {users.map(u => <li key={u.id}>{u.name}</li>)}
  </ul>
);
}

```

依存配列の動き: - [] = 初回のみ - [count] = count が変わるたび - 指定なし = 毎回（普通使わない）

2.4 リスト描画と key

配列を .map で JSX に変換するときは **必ず key** を付ける。

```

{users.map(u => (
  <div key={u.id}>{u.name}</div>
))}

```

2.5 条件分岐

```
{isLoading && <p>読み込み中...</p>}
```

```
{user ? <p>ようこそ {user.name}</p> : <p>ログインしてね</p>}
```

```

{count > 10 ? (
  <p>多すぎ</p>
) : count > 0 ? (
  <p>{count}個</p>
) : (
  <p>0 個</p>
)}

```

2.6 イベントハンドラ

```
<button onClick={() => console.log("clicked")}>押す</button>
```

```

<input
  type="text"
  value={name}
  onChange={(e) => setName(e.target.value)}
/>

```

```

<form onSubmit={(e) => { e.preventDefault(); doSomething(); }}>
  ...
</form>

```

2.7 カスタムフック

ロジックを再利用する。use で始まる関数。

```
function useTimer(intervalMs: number) {
  const [count, setCount] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setCount(c => c + 1), intervalMs);
    return () => clearInterval(id); // クリーンアップ
  }, [intervalMs]);
  return count;
}

// 使う側
const tick = useTimer(1000);
```

第 3 部担当範囲と作成物

3.1 担当範囲

ファイル	やること
frontend/src/components/PresenceList.tsx	在室一覧の見た目を整える
frontend/src/components/Character.tsx	kuremoto の素材に差し替え
frontend/src/components/ManualCheckin.tsx	UI 改善、トースト風通知
frontend/src/hooks/usePresencePing.ts	エラー時のリトライ、タブ非表示時の調整
frontend/src/styles.css	全体の見た目
frontend/public/icon-192.png, icon-512.png	PWA アイコン (kuremoto から)
frontend/vite.config.ts	manifest 調整

3.2 完成イメージ

1. ホーム画面に「現在 3 人在室中」と表示
2. キャラクター画像 (kuremoto 制作) が並び、在室時間で見た目が変わる
3. 不在の人はグレーアウト
4. 手動チェックイン/チェックアウトボタンが動く
5. PWA としてスマホのホーム画面に追加できる

第 4 部 実装手順

ステップ 0: ブランチを切る

```
git switch main
git pull origin main
git switch -c feature/takebayashi_presence-ui
```

ステップ 1: 既存コードを読む

まず以下を読んで全体構造を理解:

```
frontend/src/
├── App.tsx                # ルート
├── main.tsx               # エントリ
├── styles.css             # CSS
├── api/client.ts          # API クライアント
├── hooks/usePresencePing.ts # 1分 ping
├── components/
│   ├── PresenceList.tsx   # 在室一覧（あなたの主戦場）
│   ├── Character.tsx      # キャラ 1 人分
│   └── ManualCheckin.tsx  # 手動切替
├── pages/Login.tsx
└── types.ts
```

`npm run dev` で動かしながら、ブラウザ DevTools で各コンポーネントの動きを見る。

ステップ 2: PresenceList を整える

`frontend/src/components/PresenceList.tsx` を改善:

```
import { useEffect, useState } from "react";
import { api } from "../api/client";
import type { PresenceView } from "../types";
import { Character } from "../Character";

export function PresenceList({ refreshMs = 15_000 }: { refreshMs?: number }) {
  const [list, setList] = useState<PresenceView[]>([]);
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let cancelled = false;
```

```

const fetchOnce = async () => {
  try {
    const data = await api.listPresences();
    if (cancelled) return;
    setList(data);
    setError(null);
  } catch (e) {
    if (!cancelled) setError(String(e));
  } finally {
    if (!cancelled) setLoading(false);
  }
};

fetchOnce();
const id = setInterval(fetchOnce, refreshMs);
return () => {
  cancelled = true;
  clearInterval(id);
};
}, [refreshMs]);

const presentList = list.filter(p => p.isPresent);
const absentList = list.filter(p => !p.isPresent);

if (loading) return <section className="card"><p>読み込み中...</p></section>;

return (
  <>
    <section className="card">
      <div className="row">
        <h2 style={{ margin: 0 }}>□ 現在 {presentList.length} 人在室中</h2>
        <span className="spacer" />
        <small>{new Date().toLocaleTimeString()}更新</small>
      </div>
      {error && <p style={{ color: "crimson" }}>取得失敗: {error}</p>}
      {presentList.length === 0 ? (
        <p style={{ color: "var(--muted)" }}>誰もいないみたい...</p>
      ) : (
        <div className="character-grid" style={{ marginTop: 12 }}>
          {presentList.map(p => <Character key={p.userId} p={p} />)}
        </div>
      )}
    </section>
  </>
)

```



```

    {absentList.length > 0 && (
      <section className="card">
        <h3 style={{ marginTop: 0 }}>不在のメンバー</h3>
        <div className="character-grid">
          {absentList.map(p => <Character key={p.userId} p={p} />)}
        </div>
      </section>
    )}
  </>
);
}

```

ステップ 3: Character.tsx を画像対応に

kuremoto の素材命名: {avatarId}_{stage}.png、例: soldier-blue_lv1.png。

frontend/public/avatars/ に PNG を配置するルールにしておく。

```

import type { PresenceView } from "../types";

function stageOf(durationSec: number): "lv1" | "lv2" | "lv3" | "lv4" {
  const hours = durationSec / 3600;
  if (hours < 1) return "lv1";
  if (hours < 3) return "lv2";
  if (hours < 6) return "lv3";
  return "lv4";
}

export function Character({ p }: { p: PresenceView }) {
  const stage = stageOf(p.durationSec);
  const src = `/avatars/${p.user.avatarId}_${stage}.png`;
  const minutes = Math.floor(p.durationSec / 60);

  return (
    <div className={`character ${p.isPresent ? "present" : "absent"}`} >
      <img
        src={src}
        alt={p.user.name}
        className="avatar-img"
        onError={(e) => { e.currentTarget.src = "/avatars/fallback.png"; }}
      />
      <div className="name">{p.user.name}</div>
    </div>
  );
}

```

```

    <div className="status">
      {p.isPresent ? `在室 ${minutes}分` : "不在"}
      {p.source === "manual" && p.isPresent ? " (手動) " : ""}
    </div>
  </div>
);
}

```

CSS:

```

.avatar-img {
  width: 96px;
  height: 96px;
  object-fit: contain;
}
.character.present .avatar-img {
  animation: wiggle 1.4s ease-in-out infinite;
}

```

ステップ 4: usePresencePing にリトライを追加

frontend/src/hooks/usePresencePing.ts:

```

import { useEffect } from "react";
import { api } from "../api/client";

export function usePresencePing(enabled: boolean, intervalMs = 60_000) {
  useEffect(() => {
    if (!enabled) return;
    let cancelled = false;

    const send = async (retries = 2): Promise<void> => {
      try {
        await api.ping();
      } catch (e) {
        console.warn("[ping] failed", e);
        if (retries > 0 && !cancelled) {
          await new Promise(r => setTimeout(r, 3000));
          return send(retries - 1);
        }
      }
    }

    send();
  });
}

```

```

const id = setInterval(() => {
  if (cancelled) return;
  if (document.hidden) return; // タブ非表示時はスキップ
  send();
}, intervalMs);

// タブを表示に戻したら即 ping
const onVisible = () => { if (!document.hidden) send(); };
document.addEventListener("visibilitychange", onVisible);

return () => {
  cancelled = true;
  clearInterval(id);
  document.removeEventListener("visibilitychange", onVisible);
};
}, [enabled, intervalMs]);
}

```

ステップ 5: ManualCheckin の改善

トースト風の通知に。一定時間で消える。

```

import { useState } from "react";
import { api } from "../api/client";

export function ManualCheckin() {
  const [busy, setBusy] = useState<"checkin" | "checkout" | null>(null);
  const [toast, setToast] = useState<string>("");

  const showToast = (msg: string) => {
    setToast(msg);
    setTimeout(() => setToast(""), 3000);
  };

  const handle = async (action: "checkin" | "checkout") => {
    setBusy(action);
    try {
      await api.manual(action);
      showToast(action === "checkin" ? "☑ 在室にしました" : "☐ 退室にしました");
    } catch (e) {
      showToast(`⚠ 失敗: ${e}`);
    } finally {
      setBusy(null);
    }
  };
}

```

```

    }
  };

  return (
    <section className="card">
      <h2 style={{ marginTop: 0 }}>手動切替</h2>
      <p style={{ marginTop: 0, color: "var(--muted)" }}>
        Wi-Fi 判定が効かないとき（出張先、4G 接続時など）の補助。
      </p>
      <div className="row">
        <button className="primary" disabled={busy !== null} onClick={() => handle
          {busy === "checkin" ? "...": "☒ チェックイン"}}
        </button>
        <button disabled={busy !== null} onClick={() => handle("checkout")}>
          {busy === "checkout" ? "...": "☐ チェックアウト"}
        </button>
      </div>
      {toast && (
        <div style={{
          marginTop: 12, padding: "8px 12px",
          background: "var(--primary)", color: "white",
          borderRadius: 8, fontSize: 14,
        }}>
          {toast}
        </div>
      )}
    </section>
  );
}

```

ステップ 6: PWA アイコンを配置

kuremoto から PNG を受け取ったら frontend/public/ に配置:

- ・ icon-192.png (192x192px)
- ・ icon-512.png (512x512px)

frontend/vite.config.ts の manifest 部分は既に設定済み。

スマホで <http://your-deployed-url> を開いて「ホーム画面に追加」できることを確認。

ステップ 7: スタイルの仕上げ

frontend/src/styles.css を整える。レスポンス対応:

```
@media (max-width: 600px) {  
  #root {  
    padding: 8px;  
  }  
  header {  
    flex-direction: column;  
    align-items: flex-start;  
    gap: 8px;  
  }  
  .character-grid {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

ステップ 8: 動作確認

1. backend と frontend を起動
2. ブラウザで `http://localhost:5173` を開く
3. ログイン
4. 在室一覧が表示される
5. 「手動チェックイン」を押すと自分が在室に → 数秒後に画面に反映
6. 「手動チェックアウト」で不在に
7. DevTools の Network タブで `/api/presence/ping` が 1 分ごとに飛んでるか確認
8. スマホでアクセスして見た目チェック

ステップ 9: PR を出す

```
git add frontend/  
git commit -m "feat: 在室一覧の表示改善、キャラ画像対応、ping リトライ追加"  
git push -u origin feature/takebayashi_presence-ui
```

第 5 部よくある詰まりどころ

画面が真っ白

DevTools の Console を見る。だいたい: - 構文エラー (TSX の閉じタグ忘れ等) - import パスのタイプ - props の型不一致

API リクエストが 401

token が無いか、期限切れ。localStorage.clear() して再ログイン。

画像が表示されない

- ・ frontend/public/avatars/ にファイルが置いてあるか
- ・ のパスが/avatars/... で始まっているか
- ・ 大文字小文字が一致しているか

useEffect が無限ループ

依存配列を確認。state を変更するなら依存に含めない。

// ☒ 無限ループ

```
useEffect(() => {  
  setCount(count + 1);  
}, [count]);
```

// ☑

```
useEffect(() => {  
  const id = setInterval(() => setCount(c => c + 1), 1000);  
  return () => clearInterval(id);  
}, []);
```

CSS が効かない

- ・ クラス名のタイプ
- ・ 別のスタイルで上書きされている → DevTools の Computed タブで確認

第 6 部ここまでやったら他の人と合流

マイルストーン	誰と何を
ステップ 2 完了	tsutsumi: API が想定の形式で返ってきているか確認
ステップ 3 完了	kuremoto: 素材の命名と段階数を確定
ステップ 4 完了	tsutsumi: ping のタイムアウトが噛み合うか確認
ステップ 6 完了	kuremoto: PWA アイコンの素材を受け取る
PR マージ後	全員: モバイルで動作確認

付録 A: チートシート

開発

```
cd frontend && npm run dev
```

型エラーだけチェック

```
cd frontend && npx tsc --noEmit
```

ビルド (本番形式の確認)

```
cd frontend && npm run build
```

```
cd frontend && npm run preview # ビルド結果を見る
```

// React パターン早見表

```
const [x, setX] = useState(0);
useEffect(() => { /* ... */ }, [deps]);
```

```
{condition && <Component />}
{flag ? <A /> : <B />}
{items.map(i => <Item key={i.id} {...i} />)}
```

```
<button onClick={() => doSomething()}>click</button>
<input value={name} onChange={(e) => setName(e.target.value)} />
```

付録 B: 用語集

- ・ **JSX/TSX**: HTML を JavaScript/TypeScript の中に書ける記法
- ・ **props**: 親コンポーネントから渡される引数
- ・ **state**: コンポーネント内で変化する値
- ・ **hook**: use で始まる React の関数 (useState, useEffect, ...)
- ・ **依存配列**: useEffect の []. 何が変わったら再実行するかを指定
- ・ **クリーンアップ**: useEffect が return する関数。タイマー解除など

- ・ 再描画 (re-render): state が変わったとき React が画面を更新すること

この指南書を一通り終わると、在室画面ができあがります。困ったらすぐ LINE で相談。